Reinforcement Learning Summary

Mattiahs Bubich

April 15, 2025

Contents

1 Introduction to Reinforcement Learning **4**
1.1 Exploration vs Exploitation . . . 4
1.2 Goal-Directed Interaction . . . 4
1.3 Key Elements of RL Systems . . . 4
1.4 Models and Planning . . . 4
1.5 Learning vs Evolutionary Methods . . . 4
1.6 Computational Perspective . . . 5
1.7 Markov Decision Process Framework . . . 5

2 Finite Markov Decision Processes **6**
2.1 Sequential Decision Making and Credit Assignment . . . 6
2.2 Interaction Over Time . . . 6
2.3 The Role of Policy . . . 6
2.4 The Return . . . 7
2.5 Value Functions . . . 7
2.6 Knowledge Assumptions . . . 8
2.7 Limitations in Practice . . . 8

3 David Silver: Introduction to Reinforcement Learning **9**
3.1 Overview . . . 9
3.2 Core Concepts . . . 9
3.3 Applications . . . 9

4 OpenAI Spinning Up Article **10**
4.1 Key Concepts in RL . . . 10
4.2 Kinds of RL Algorithms . . . 11
4.3 Intro to Policy Optimization . . . 12

5 Reinforcement Learning in the Context of Machine Learning **13**
5.1 How RL Relates to Other ML Paradigms . . . 13
5.2 The Complexity of RL . . . 13
5.3 A Hierarchy of Learning Problems . . . 13
5.4 Special Case: Baseline Distribution RL . . . 14

6 Reinforcement Learning in Robotics **15**
6.1 Challenges and Considerations . . . 15
6.2 Perspective and Goals . . . 15

7 Reinforcement Learning in the Context of Optimal Control **17**
7.1 Relationship Between RL and Optimal Control . . . 17
7.2 Key Differences Between RL and Optimal Control . . . 17
7.3 RL as Adaptive Optimal Control . . . 17

8 Challenges in Robot Reinforcement Learning **18**
8.1 Overview of Challenges . . . 18
8.2 Curse of Dimensionality . . . 18
8.3 Addressing Dimensionality . . . 19
8.4 Curse of Under-Modeling and Model Uncertainty . . . 19
8.5 Curse of Goal Specification . . . 20

9 Reinforcement Learning in Robotics: Applications and Real-World Challenges **22**
9.1 Introduction . . . 22

10 State-of-the-Art Reinforcement Learning Algorithms in Robotics **23**
10.1 Policy-Search and Modern Algorithms . . . 23

11 Challenges for the Policy Representation in Robotics **25**
11.1 Challenges for the Policy Representation in Robotics . . . 25

12 Perspectives on Sim2Real Transfer for Robotics **27**
12.1 Core Questions and Debates . . . 27
12.2 Panel Discussion Insights . . . 27
12.3 Research Contributions . . . 27
12.4 Conclusions and Future Directions . . . 28

13 Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World **29**
13.1 Motivation . . . 29
13.2 Key Idea: Domain Randomization . . . 29
13.3 Method . . . 29
13.4 Results . . . 29
13.5 Conclusion and Future Work . . . 30

14 Sim-to-Real Transfer of Robotic Control with Dynamics Randomization **31**
14.1 Motivation . . . 31
14.2 Method . . . 31
14.3 Training Approach . . . 31
14.4 Architecture . . . 31
14.5 Results . . . 32
14.6 Conclusion . . . 32

15 Pretraining Language Models with Human Preferences **33**
15.1 Motivation and Background . . . 33
15.2 Key Contributions . . . 33
15.3 Limitations and Challenges . . . 33
15.4 Future Directions . . . 34
15.5 Conclusion . . . 34

16 Generalizing from Simulation (OpenAI) 35
16.1 Core Idea . . . 35
16.2 Key Techniques . . . 35
16.3 Experiments and Results . . . 35
16.4 Implications . . . 35

17 Randomization and the Reality Gap: Improving Policy Transfer in Sim-to-Real Reinforcement Learning 36
17.1 Overview . . . 36
17.2 Motivation and Problem Statement . . . 36
17.3 Domain Randomization Approach . . . 36
17.4 Experiments . . . 36
17.5 Findings and Insights . . . 37
17.6 Practical Recommendations . . . 37
17.7 Conclusion . . . 37

18 Domain Randomization for Sim2Real Transfer (Lilian Weng, 2019) 38
18.1 Motivation . . . 38
18.2 Sim2Real Transfer Strategies . . . 38
18.3 What is Domain Randomization? . . . 38
18.4 Uniform Domain Randomization (UDR) . . . 38
18.5 Why Domain Randomization Works . . . 38
18.6 Guided Domain Randomization . . . 39
18.7 Real Data-Guided DR and Distribution Matching . . . 39
18.8 Simulator-Guided Domain Randomization . . . 39
18.9 Conclusion . . . 39

1 Introduction to Reinforcement Learning

Definition

Reinforcement learning is learning what to do—how to map situations to actions—so as to maximize a numerical reward signal. The learner is not told which actions to take, but instead must discover which actions yield the most reward by trying them.

1.1 Exploration vs Exploitation

One of the challenges that arise in reinforcement learning, and not in other kinds of learning, is the trade-off between **exploration** and **exploitation**. To obtain a lot of reward, an agent must prefer actions that it has tried in the past and found effective. But to discover those actions, it must try others that it hasn't selected before. The agent must exploit known strategies while still exploring to improve future decision-making.

1.2 Goal-Directed Interaction

Another key feature of reinforcement learning is that it explicitly considers the full problem of a **goal-directed agent interacting with an uncertain environment**.

1.3 Key Elements of RL Systems

Beyond the agent and the environment, a reinforcement learning system includes four main subelements:

- **Policy**: defines the agent's behavior at a given time.
- **Reward signal**: defines the goal by providing feedback (a scalar number) at each time step.
- **Value function**: evaluates the desirability of states in terms of expected future rewards.
- **Model of the environment (optional)**: predicts next states and rewards, enabling planning.

Definition

Whereas the reward signal indicates what is good in an immediate sense, a value function specifies what is good in the long run.

1.4 Models and Planning

A **model of the environment** allows inferences about how the environment behaves. Given a state and action, the model might predict the resulting next state and reward. Models are used for **planning**—choosing actions based on anticipated outcomes before they are experienced.

1.5 Learning vs Evolutionary Methods

Our focus is on reinforcement learning methods that learn while interacting with the environment. **Evolutionary methods** do not do this. They ignore key structure in the problem—like which states are visited or which actions are selected—and do not treat the policy as a mapping from states to actions.

1.6 Computational Perspective

Reinforcement learning is a **computational approach to understanding and automating goal-directed learning and decision making**.

1.7 Markov Decision Process Framework

Reinforcement learning uses the formal framework of **Markov Decision Processes (MDPs)** to define the interaction between the learning agent and its environment. This framework captures:

- Cause and effect
- Uncertainty and nondeterminism
- Explicit goals

2 Finite Markov Decision Processes

2.1 Sequential Decision Making and Credit Assignment

In this chapter we introduce the formal problem of **finite Markov decision processes (MDPs)**, which we will try to solve throughout the rest of the book.

- This problem involves **evaluative feedback**, as in multi-armed bandit problems.
- It also introduces an **associative aspect**—choosing different actions in different situations.

MDPs are a classical formalization of **sequential decision making**, where actions influence not only immediate rewards but also future states and, indirectly, future rewards.

This introduces:

- **Delayed reward**
- The need to **trade off immediate vs. future reward**

Definition

In bandit problems, we estimate the value $q^*(a)$ of each action a. In MDPs, we estimate either:

- $q^*(s, a)$: the value of each action in each state, or
- $v^*(s)$: the value of each state under optimal action selection

These state-dependent values are key to assigning credit for long-term consequences.

2.2 Interaction Over Time

Reinforcement learning is about learning from **interaction** how to behave in order to achieve a goal.

- The agent and environment interact over a sequence of discrete time steps.
- Their interface defines a task:
  - **States**: the basis for making choices
  - **Actions**: the choices made by the agent
  - **Rewards**: the evaluation of those choices

Note

Everything inside the agent is known and controllable; the environment is partially unknown and only indirectly controllable.

2.3 The Role of Policy

A **policy** is a stochastic rule by which the agent selects actions based on the current state.

Objective: Maximize the total reward received over time.

When the above setup includes well-defined transition probabilities, it forms a **Markov Decision Process (MDP)**. A **finite MDP** has finite sets of:

- States S

- Actions A
- Rewards R

While the theory often focuses on finite MDPs, most concepts generalize beyond them.

2.4 The Return

The **return** is a function of future rewards that the agent seeks to maximize (in expectation). It has multiple definitions depending on whether we wish to **discount delayed reward**.

- **Undiscounted return** is used in **episodic tasks** (natural episodes).
- **Discounted return** is used in **continuing tasks** (no natural end).

The return is typically defined as:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

Definition

Discount factor γ: The scalar $\gamma \in [0, 1]$ determines how much the agent values future rewards compared to immediate ones.

- $\gamma = 0$: only immediate rewards are considered.
- $\gamma \to 1$: future rewards are considered nearly as important as immediate ones.

Choosing γ balances the agent's short-term vs. long-term priorities and reflects the uncertainty or importance of long-term outcomes.

2.5 Value Functions

Definition

A **value function** assigns expected return to:

- Each state (state value function $v_\pi(s)$)
- Each state–action pair (action value function $q_\pi(s, a)$)

Optimal value functions:

- $v^*(s) = \max_\pi v_\pi(s)$
- $q^*(s, a) = \max_\pi q_\pi(s, a)$

A policy π^* is **optimal** if:

$$v_{\pi^*}(s) = v^*(s), \quad \forall s \in S$$

2.6 Knowledge Assumptions

The formulation of a reinforcement learning problem can vary depending on the assumptions about the agent's prior knowledge.

- **Complete knowledge**: the agent has a full, accurate model of the environment.
  - If modeled as an MDP, this includes a four-argument dynamics function $p(s', r \mid s, a)$.
- **Incomplete knowledge**: the agent lacks a perfect model of the environment.

2.7 Limitations in Practice

Even with a complete environment model, the agent may struggle to use it due to:

- **Memory limitations**
- **Computational limitations**
- **State-space complexity** — the number of states is often too large for exact tabular methods.

In practice, approximation techniques are crucial to scale reinforcement learning to real-world tasks.

3 David Silver: Introduction to Reinforcement Learning

This section summarizes the key concepts from the video lecture *"Introduction to Reinforcement Learning" by David Silver*.

3.1 Overview

Reinforcement learning (RL) is introduced as a computational approach to decision making based on interaction with an environment. An agent learns to make sequential decisions in order to maximize cumulative rewards.

3.2 Core Concepts

- **Agent–Environment Interaction:** The agent observes the state, selects an action, receives a reward, and transitions to a new state — forming a feedback loop over discrete time steps.
- **Exploration vs. Exploitation:** The agent must balance:
  - **Exploration** – trying new actions to gather information
  - **Exploitation** – selecting actions that yield the highest known reward
- **Markov Decision Processes (MDPs):** MDPs formalize the RL problem as a tuple (S, A, P, R, γ) involving states, actions, transition probabilities, rewards, and discounting.
- **Policies:** A policy $\pi(a|s)$ is a mapping from states to a probability distribution over actions. It defines the agent's behavior.
- **Value Functions:**
  - $v_\pi(s)$: expected return from state s under policy π
  - $q_\pi(s, a)$: expected return from state–action pair (s, a) under policy π
- **Model-Based vs. Model-Free:**
  - **Model-based** methods learn a model of the environment to make predictions and plan.
  - **Model-free** methods learn value functions or policies directly from experience.

3.3 Applications

David Silver outlines the vast applicability of RL across domains such as:

- Robotics
- Game playing (e.g., AlphaGo)
- Autonomous systems
- Control and decision-making tasks

Note

This lecture establishes the foundation of reinforcement learning as a field and prepares the ground for more advanced topics such as dynamic programming, Monte Carlo methods, and temporal-difference learning.

4 OpenAI Spinning Up Article

4.1 Key Concepts in RL

This section summarizes the core ideas introduced in OpenAI's Spinning Up guide to reinforcement learning. (Source: Spinning Up – RL Introduction)

- **Reinforcement Learning (RL)** is a way for agents to learn behavior by interacting with an environment and receiving feedback in the form of rewards or penalties. The goal is to learn which actions lead to high cumulative reward.
- **States and Observations:** A *state* is a full description of the world. An *observation* is partial and may hide some information. If the agent sees everything, the environment is *fully observed*; otherwise, it is *partially observed*.
- **Action Spaces:** The set of all possible actions is the *action space*.
  - **Discrete:** Finite options (e.g., move left/right).
  - **Continuous:** Real-valued actions (e.g., robot angles).
- **Policies:** A policy is the agent's rule for choosing actions.
  - *Deterministic:* Always the same action for a state.
  - *Stochastic:* Picks actions with certain probabilities.

  In deep RL, policies are often implemented as neural networks with parameters.
- **Trajectories:** A *trajectory* is a sequence of states and actions:

$$\tau = (s_0, a_0, s_1, a_1, \dots)$$

- **Reward and Return:** The agent receives a reward after each action. The *return* is the total sum of rewards received.
  - *Undiscounted return:* Finite sum over steps.
  - *Discounted return:* Uses factor $\gamma \in (0, 1)$ to value immediate rewards more.
- **Value Functions:**
  - $V^\pi(s)$: Expected return starting from state s using policy π
  - $Q^\pi(s, a)$: Expected return from state s, taking action a, then following π
- **Bellman Equations:** These recursive equations express the value of a state in terms of expected rewards and the value of next states. They are essential to many RL algorithms.
- **Advantage Function:** Measures how much better an action is compared to the average:

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$$

- **The RL Problem:** The main goal is to find a policy π^* that maximizes expected return over all possible trajectories:

$$\pi^* = \arg\max_\pi \mathbb{E}_{\tau\sim\pi}[R(\tau)]$$

4.2 Kinds of RL Algorithms

This section summarizes the second part of OpenAI's Spinning Up guide, which explores the landscape of modern RL algorithms and the trade-offs involved in their design. (Source: Spinning Up – Kinds of RL Algorithms)

- **Model-Free vs Model-Based RL**
  - **Model-Free:** Does not use a model of the environment. Learns directly from interactions. Easier to implement and tune. *Examples: DQN, PPO*
  - **Model-Based:** Uses (or learns) a model to simulate the environment. Enables planning, can be more sample-efficient. *Example: AlphaZero*
- **What Can Be Learned?**
  - Policies (stochastic or deterministic)
  - Value functions
  - Q-functions
  - Models of the environment
- **Model-Free RL Methods**
  - **Policy Optimization:**
    - Directly optimizes a policy $\pi_\theta(a|s)$
    - Usually requires *on-policy* data
    - Often includes a learned value function $V_\phi(s)$
    - *Examples: A2C, A3C, PPO*
  - **Q-Learning:**
    - Learns a Q-function $Q_\theta(s, a)$
    - Uses the Bellman equation
    - Can work *off-policy*
    - *Examples: DQN, C51*
  - **Trade-offs:**
    - Policy optimization is more stable, direct
    - Q-learning is more data-efficient but less stable
- **Hybrid Methods (in between)**
  - Combine Q-learning and policy optimization techniques
  - *Examples:*
    - DDPG: learns both a policy and Q-function
    - SAC: uses stochastic policies with entropy regularization
- **Model-Based RL Techniques**
  - **Pure Planning (MPC):** Computes a full action plan at each step using the model.
  - **Expert Iteration:** Uses planning to improve the policy over time. *Examples: ExIt, AlphaZero*
  - **Data Augmentation:** Mixes real experience with imagined data. *Examples: MBVE, World Models*

– **Embedded Planning in Policies:** Planning becomes part of the policy, learned when and how to use it. *Example: I2A (Imagination-Augmented Agents)*

Note

This taxonomy highlights key algorithmic choices in RL: whether to use a model, what to learn, and how to balance between stability and efficiency.

4.3 Intro to Policy Optimization

This section introduces the core ideas behind **policy gradient methods**, which are a family of algorithms that optimize a policy directly to improve an agent's performance.

- **Objective:** Maximize expected return $J(\pi_\theta) = \mathbb{E}_{\tau\sim\pi_\theta}[R(\tau)]$
- **Policy Gradient:** The gradient of expected return with respect to policy parameters. Computed as:
$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t) R(\tau)\right]$$
- **Implementation:** Using neural networks to define the policy and compute gradients via PyTorch's `Categorical` distribution tools.
- **Reward-to-Go:** Instead of using the total episode return $R(\tau)$ at every time step, we use only the **future rewards**:
$$\hat{R}_t = \sum_{t'=t}^{T} r_{t'}$$
This reduces variance in the policy gradient estimate.
- **Baselines:** A baseline $b(s_t)$, often the value function $V^\pi(s_t)$, can be subtracted to further reduce variance:
$$\nabla_\theta J(\pi_\theta) = \mathbb{E}\left[\sum_t \nabla_\theta \log \pi_\theta(a_t|s_t)\left(\hat{R}_t - b(s_t)\right)\right]$$
- **Advantage Function:** Measures how much better an action is compared to average:
$$A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$$
and can be used in the policy gradient:
$$\Phi_t = A^\pi(s_t, a_t)$$
- **Expected Grad-Log-Prob Lemma (EGLP):**
$$\mathbb{E}_{x\sim P_\theta}[\nabla_\theta \log P_\theta(x)] = 0$$
This is used to justify ignoring past rewards or adding baselines.

Note

Policy gradients are central to modern RL. By using reward-to-go and baselines (like value functions or advantage functions), we can reduce the variance in learning and improve convergence speed.

5 Reinforcement Learning in the Context of Machine Learning

5.1 How RL Relates to Other ML Paradigms

Reinforcement Learning (RL) can be understood more clearly by comparing it with other machine learning paradigms.

- **Supervised Learning:** The agent receives labeled examples with the correct outputs. There is no interaction; each input–output pair is independent. *Example:* Face detection, spam filtering.
- **Imitation Learning:** The agent observes demonstrations from an expert. Although "correct" actions are given, the interaction is sequential: one mistake alters future observations. *Example:* A robot mimicking a human pilot.
- **Contextual Bandits:** The agent chooses an action based on context but receives feedback only for that action — not for alternatives. It introduces the **exploration vs exploitation** dilemma. *Example:* Web ad placement, drug discovery.
- **Structured Prediction:** Similar to imitation learning, it involves making multiple interdependent predictions. *Example:* Parsing natural language, object detection in images.

5.2 The Complexity of RL

Reinforcement learning combines both:

- **Sequential Interaction** — like imitation learning
- **Sparse/partial feedback** — like contextual bandits

This dual complexity makes RL powerful but also:

- **Hard to solve** (computationally and information-theoretically)
- **General:** RL can be used to solve many problems that simpler learning paradigms cannot.

5.3 A Hierarchy of Learning Problems

A conceptual framework organizes learning problems based on:

- **Interaction complexity**
- **Reward structure complexity**

Reinforcement learning *subsumes* simpler problems — meaning those problems can be framed within the RL framework.

Note

Reduction techniques can convert supervised learning solutions into RL solutions, and vice versa. This modularity is useful in practice.

5.4 Special Case: Baseline Distribution RL

A variant of RL where the agent can start from states sampled from an expert distribution (rather than random initial states). This can drastically simplify learning, especially when it's important to focus on specific regions of the state space.

—

RL's richness and difficulty stem from this combination of:

- Decision-making over time
- Sparse and delayed feedback
- A need for exploration

These characteristics make it especially relevant (and challenging) for robotics.

6 Reinforcement Learning in Robotics

6.1 Challenges and Considerations

Reinforcement learning in robotics presents unique challenges compared to standard RL benchmarks. These challenges stem from the nature of robotic environments and the constraints of real-world data collection.

- **High-Dimensional, Continuous Spaces:** Robotic systems often involve continuous state and action spaces with many dimensions (10–30+), which increases complexity.
- **Partial Observability:** Robots rarely have full access to the true state of the environment. Instead, they rely on noisy observations and need filtering techniques (e.g., Kalman filters) to estimate the underlying state and associated uncertainty.
- **Expensive Real-World Interaction:** Collecting experience on physical robots is costly, slow, and often non-reproducible. For example, reproducing identical initial conditions in tasks like robot table tennis is impractical.
- **Need for Approximation:** Due to limited experience, learning systems must approximate:
  - The state representation
  - Policies
  - Value functions
  - Environment dynamics
- **Limitations of Simulation:** Simulated environments can help, but even small model errors can accumulate and degrade performance in the real world (especially in dynamic tasks). Algorithms must be robust to *under-modeling* and uncertainty.
- **Reward Shaping:** Carefully designing the reward function is crucial for effective learning. Good rewards guide learning faster but often require significant domain knowledge. This manual tuning can be difficult and is a major practical barrier.
- **Algorithm Choice:** Not all RL methods are suitable for robotics.
  - Model-based approaches are common and effective.
  - Policy search methods are often preferred over value-based methods.
- **Useful Strategies for Robotics RL:**
  - Using expressive policy/value representations
  - Incorporating prior knowledge
  - Transferring knowledge from simulation to reality

6.2 Perspective and Goals

- The paper surveys a broad range of robotics tasks where RL has been applied.
- If a task has temporal structure and can be formulated as an optimization problem, RL is often applicable.
- It aims to:
  - Guide the robotics community on what RL can achieve

- Highlight promising approaches
- Provide the RL community with real-world challenges and new research opportunities

Note

The authors emphasize that robotics-specific constraints require adapted RL methods. Key insights include the use of models, policy search, reward shaping, and leveraging prior or simulated experience to handle the cost and complexity of real-world learning.

7 Reinforcement Learning in the Context of Optimal Control

7.1 Relationship Between RL and Optimal Control

Reinforcement Learning (RL) is closely related to **classical optimal control theory**, as well as other areas such as:

- Dynamic programming
- Stochastic programming
- Simulation-optimization
- Stochastic search
- Optimal stopping

Shared Goal: Both RL and optimal control seek to find an *optimal policy* (also called a controller) that maximizes or minimizes a cumulative objective — typically expressed as total reward or cost.

7.2 Key Differences Between RL and Optimal Control

- **Model Assumptions:**
  - **Optimal control** assumes perfect knowledge of the system dynamics, i.e., a known function $T(s, a)$ that predicts the next state.
  - **Reinforcement learning** does *not* assume access to a known model — it learns directly from interaction and feedback.
- **Robustness to Complexity:**
  - Classical control provides strong theoretical guarantees.
  - These guarantees often break down in practice due to:
    - Inaccuracies in the model
    - Computational complexity
  - RL handles such limitations with *approximation and data-driven methods.*

7.3 RL as Adaptive Optimal Control

RL can be seen as a form of **adaptive optimal control**:

- It adapts to unknown dynamics by learning from trial-and-error.
- It often integrates ideas from control theory, such as:
  - Linear-Quadratic Regulation (LQR)
  - Differential Dynamic Programming (DDP)
- These classical techniques can be applied to *learned system models* through repeated environment interaction.

Note

In robotics, a major line of RL research combines optimal control tools with data-driven model learning to balance theory and flexibility. This hybrid perspective enables tractable, real-world robot learning.

8 Challenges in Robot Reinforcement Learning

8.1 Overview of Challenges

Reinforcement learning in robotics introduces specific and often more severe challenges compared to classical RL settings. These challenges arise due to the physical nature of robots and their need for real-time, data-efficient learning.

- **Continuous State and Action Spaces:** Robots operate in high-dimensional continuous domains. This requires careful choices for:
  - Control granularity
  - Discretization or function approximation
  - Time step resolution
- **Real-World Sampling is Expensive:** Trials (roll-outs) require time, may involve human intervention, and wear down hardware. This limits the amount of experience available for learning.
- **Uncertainty and Noise:** Physical systems exhibit unpredictable behavior, requiring algorithms to handle sensor noise, delays, and model inaccuracies.
- **Real-Time Constraints:** Algorithms must work under sensing and actuation delays, and be capable of running in real time on embedded systems.
- **Simulation Limitations:** Simulations help reduce costs, but real-world deployment demands robustness to simulation inaccuracies (model errors).
- **Reward Specification:** Defining good reward functions is essential and challenging. Poorly designed rewards can make learning infeasible due to ineffective exploration.

8.2 Curse of Dimensionality

Definition: As defined by Bellman (1957), the *Curse of Dimensionality* refers to the exponential growth of the state-action space with each additional dimension. For example:

With 10 discretization levels per dimension, an n-dimensional space has 10^n states.

In Robotics: Robots frequently operate in state spaces with 20+ dimensions and continuous action spaces. *Example:* In a ball-paddling task:

- 7 joint angles + velocities
- Ball position + velocity
- = 20-dimensional state; 7-dimensional action (torques)

Practical Implications:

- Full state coverage is computationally infeasible.
- Hierarchical decomposition is often used to simplify tasks.
- Embedded control systems may encapsulate low-level complexity (e.g., motion primitives or trajectory fragments).
- This simplification, however, limits the robot's full dynamic potential.

8.3 Addressing Dimensionality

The RL community offers a variety of tools to combat the curse of dimensionality:

- **Adaptive Discretization** — dynamically adjusts resolution where needed.
- **Function Approximation** — generalizes over large spaces (e.g., neural networks, linear basis functions).
- **Options Framework (Macro-Actions)** — Allows hierarchical structuring by decomposing tasks into reusable sub-tasks (e.g., open a door, move to location).
  - Reduces planning horizon
  - Facilitates transfer and modularity

Note

Dimensionality reduction and hierarchical abstractions are key to making RL feasible for real-world robotics. However, finding or learning these abstractions automatically remains a major open research area.

8.4 Curse of Under-Modeling and Model Uncertainty

One approach to reduce the need for costly real-world data is to use simulation. In theory, if the model is accurate, a robot could learn entirely in simulation and transfer the resulting policy to the real world.

The Challenge of Accurate Modeling

- **Accurate simulators are hard to build:**
  - Realistic models require large amounts of data.
  - Even small modeling errors can accumulate over time.
  - Simulated behaviors can diverge significantly from real-world outcomes.
- **Limited transferability:** Policies trained on imperfect simulators often fail when transferred. *Example:* Atkeson (1994) encountered this issue with a pendulum swing-up task.
- **Better real-world performance:** In many cases, learning directly on the real robot yields better performance due to the richness of physical interactions, such as:
  - Contact dynamics
  - Friction
  - Elastic deformations

Stability Affects Transferability

- **Stable tasks:**
  - Self-stabilizing systems (e.g., ball attached to a racket with an elastic string) can tolerate model errors better.
  - Real-world performance often exceeds expectations from simulation.
- **Unstable tasks:**
  - Tasks requiring constant control (e.g., pole balancing) are sensitive to small deviations.
  - Transferred policies often fail in these settings.

Despite Limitations, Models Are Still Useful

Approximate models, even if imperfect, are valuable tools:

- For **verifying and testing** algorithms in simulation
- To **estimate gradients** for policy optimization
- To **evaluate proximity to optimal solutions**
- To guide data collection strategies
- For **mental rehearsal** (offline policy updates and planning)

Note

While simulation is a powerful tool for reducing the cost of real-world experiments, transferring learned policies reliably requires either accurate models or robust learning strategies that can handle modeling errors.

8.5 Curse of Goal Specification

In reinforcement learning, behavior is defined implicitly through the reward function. While this abstraction simplifies design, specifying a **good reward function** is often surprisingly difficult in practice, especially in robotics.

Challenges in Reward Design

- **Lack of Informative Signal:** If the reward is too sparse (e.g., only given at task success), the agent may never receive meaningful feedback to improve.
- **Need for Intermediate Rewards:** To encourage learning, it is often necessary to **shape** the reward with intermediate signals that guide the agent toward the goal (Laud, 2004).
- **Trade-offs Between Objectives:** High reward may conflict with safety or hardware constraints (e.g., striking hard in table tennis may damage the robot).
- Penalties may be added to reduce:
  - Wear on components
  - High-frequency action switching
  - Violations of physical constraints
- **Reward Exploitation:** Agents often find unexpected shortcuts to maximize reward that do not reflect the intended behavior (Ng et al., 1999).

Reward Representation and Abstraction

- The **most natural reward** often exists in a rich state/action space that is computationally intractable for learning.
- Therefore, reward functions must be **reformulated** in the lower-dimensional or abstract space in which learning takes place.
- **Example:**

  - In the ball-in-a-cup task, a binary reward (success/failure) is intuitive, but learning is enabled only through a shaped reward based on the ball's Cartesian distance and trajectory.
  - In outdoor navigation (e.g., Crusher robot), the system reasons over simplified grid-based state representations, but produces behavior that aligns with complex human preferences like safety and speed.

Inverse Reinforcement Learning (IRL)

Alternative approach: Infer the reward function from expert demonstrations.

- **Also called:** Inverse optimal control (Russell, 1998).
- **Key Idea:** Learn a reward such that the demonstrated policy appears optimal under it.
- **Milestones:**
  - Abbeel and Ng (2004): Match the expert's expected reward over features.
  - Ratliff et al. (2006b): Viewed IRL as structured prediction; proposed scalable optimization algorithms.
  - Ziebart et al. (2008): Introduced a probabilistic, robust variant of IRL for sub-optimal demonstrations.
- **Applications:** IRL has been applied to navigation, manipulation, and locomotion tasks with real robots.

Reward Complexity vs. Learning Complexity

- A more expressive reward may simplify the policy learning problem.
- Recent work (e.g., Sorg et al., 2010; Zucker and Bagnell, 2012) uses **parametrized reward functions** to derive complex policies from simpler control problems.
- This technique enables flexible policy transfer across tasks of varying complexity (e.g., solving various marble mazes).

Note

Designing the reward is itself a learning problem: overly simple rewards may hinder learning, while expressive ones may simplify it — but require careful engineering or demonstration-based inference.

9 Reinforcement Learning in Robotics: Applications and Real-World Challenges

9.1 Introduction

Empowering robots with human-like motor skills is a key goal in robotics. A promising path toward this involves enabling robots to learn autonomously, in a way that mimics human learning. Over time, three main approaches for teaching robots have emerged:

- **Direct Programming:** A low-level, manual method still used in industrial robotics. *Advantages:* Precise control. *Disadvantages:* Time-consuming, inflexible, non-scalable.
- **Imitation Learning:** The robot learns by observing demonstrations. Three common methods:
  - **Kinesthetic Teaching:** Manually guiding the robot's limbs and recording trajectories or poses. *Limitations:* Best for lightweight robots, limited DoF, lacks scalability.
  - **Teleoperation:** Remotely controlling the robot via joystick or haptic device. *Advantages:* Allows remote teaching; easier setup. *Limitations:* Limited control feedback, latency issues.
  - **Observational Learning:** Demonstrations performed by a human, recorded via vision or motion capture. *Challenges:* Solving the correspondence problem between human and robot kinematics.

  Imitation learning allows generalization from multiple demonstrations, identifying which task components are essential vs. flexible.
- **Reinforcement Learning (RL):** Learning through trial and error, guided by a reward function. *Key benefits:*
  - Learn tasks that humans cannot directly demonstrate or program.
  - Optimize unknown or non-analytical objectives (e.g., energy efficiency, speed).
  - Adapt to novel variations or environmental conditions (e.g., from flat ground to slope).
  - Refine behaviors from rough initial demonstrations.
  - Compensate for hardware changes (wear, thermal effects, physical evolution).

Comparison of Approaches

- Direct programming ensures precise control but lacks flexibility.
- Imitation learning facilitates natural interaction but struggles with speed, embodiment differences, and task coverage.
- RL allows for broader autonomy but requires careful specification of reward, policy structure, and exploration strategy.
- A future method, "goal-directed learning", might involve specifying only the goal, with the robot autonomously figuring out the rest.

Purpose and Structure of This Work

This paper presents a synthesis of empirical research and insights gathered over 4–5 years. It does not introduce new algorithms but focuses on:

- Reviewing important RL algorithms applied in robotics.
- Identifying key challenges in designing policy representations.
- Demonstrating how different policies solve distinct real-world problems through three case studies.
- Offering predictions on future directions for robot learning and emerging teaching paradigms.

Main Contribution: A deeper understanding that successful reinforcement learning in robotics depends critically on the *design of policy representations* tailored to specific challenges of robot control.

Paper Outline:

1. Overview of recent RL algorithms used in robotics
2. Challenges in policy representation
3. Description of state-of-the-art policy representations
4. Three real-world robotic task examples and solutions
5. Comparative summary of tasks and insights
6. Future perspectives on RL and robot teaching approaches

Note

While general-purpose RL algorithms are still out of reach, understanding how to represent policies effectively has proven essential for enabling real-world robot learning.

10 State-of-the-Art Reinforcement Learning Algorithms in Robotics

10.1 Policy-Search and Modern Algorithms

Robotic systems are inherently high-dimensional, involving many degrees of freedom (DoFs), continuous states and actions, and noisy sensors and actuators. These characteristics pose major challenges for traditional RL approaches based on Markov Decision Processes (MDPs), Partially Observable MDPs (POMDPs), or discretized state/action spaces—mainly due to the **curse of dimensionality**.

Policy-Search RL: A Scalable Alternative

The breakthrough in scalability came with **policy-search** reinforcement learning:

- Instead of exploring large state-action spaces, policy-search methods work in a **reduced policy space** defined by a parameterized representation.
- This drastically reduces dimensionality and improves convergence speed.
- Such methods are well-suited for continuous, high-dimensional control problems.

Policy Gradient Methods

Early successes in policy-search relied on **policy-gradient algorithms** such as:

- **Episodic Natural Actor-Critic (eNAC)**
- **Episodic REINFORCE**

These algorithms directly optimize policy parameters by estimating gradients of the expected return. However, they have limitations:

- High sensitivity to the learning rate.
- High variance due to stochastic exploration.

PoWER: EM-Based Policy Search

An alternative to gradient-based methods is **PoWER** (Policy learning by Weighting Exploration with the Returns), introduced by Kober et al.:

- Based on the Expectation-Maximization (EM) algorithm.
- **No learning rate** is required—eliminating a common tuning challenge.
- Utilizes **importance sampling** to efficiently reuse past experience.
- Demonstrates strong performance in real-robot applications.

$\mathbf{PI^2}$: Policy Improvement with Path Integrals

$\mathbf{PI^2}$, proposed by Theodorou et al., is based on stochastic optimal control:

- Uses a path integral framework to update policies.
- **Avoids numerical instabilities**—no matrix inversions or gradient learning rates required.
- Shows excellent scalability to high-dimensional problems, such as quadruped locomotion.

Stochastic Optimization Methods in RL

Several methods from stochastic optimization have also been adapted for policy search in RL:

- **Cross-Entropy Method (CEM)**
- **Covariance Matrix Adaptation Evolution Strategy (CMA-ES)**

Although originally developed outside the RL community, these algorithms:

- Offer **black-box optimization** of policies.
- Have demonstrated success in robotic skill learning tasks.
- Are gaining recognition as viable, robust alternatives for direct policy search.

Note

Policy-search algorithms—especially those based on EM, path integrals, or stochastic optimization—are among the most effective tools for applying RL in robotics, particularly in high-dimensional, real-world scenarios.

11 Challenges for the Policy Representation in Robotics

11.1 Challenges for the Policy Representation in Robotics

Having a robust policy-search reinforcement learning algorithm is essential—but not sufficient—for solving real-world robotic tasks. Before applying any RL algorithm, an appropriate **policy representation** (also called policy encoding) must be designed.

The policy representation:

- Defines the **search space** of possible behaviors the algorithm can explore.
- Influences **what can be learned**, similarly to how a hypothesis model affects the performance of a regression algorithm.
- Impacts the **performance and convergence** of the learning algorithm.

However, designing an effective policy representation is non-trivial due to the demanding nature of robotic systems. Below is a list of critical requirements and challenges that a good policy representation should ideally satisfy:

Core Requirements

- **Smoothness:** The policy must generate continuous, smooth trajectories, minimizing jerks and accelerations to ensure safety and energy efficiency. Exceptions exist (e.g., bang-bang control).
- **Safety:** The policy must respect robot-specific limits (joint angles, torques, workspace) and human safety.
- **Gradual Exploration:** It should allow for incremental, low-risk exploration. Abrupt changes in the policy can lead to unsafe or unstable behaviors.
- **Scalability:** Must scale to high-dimensional state-action spaces, as modern robots often exceed 50 DoFs.
- **Compactness:** Despite the high dimensionality, the representation should remain compact, avoiding direct use of full trajectory data as parameters.
- **Adaptability:** Must adapt to different task complexities, from heavy object manipulation to delicate tasks like microsurgery.
- **Multi-Resolution:** Different parts of the policy should allow for varying degrees of resolution and precision.
- **Unbiasedness:** Should avoid overly restricting the policy search space or requiring prior knowledge about the solution.
- **Bias/Priors:** On the flip side, it should allow injection of domain knowledge when available to guide or accelerate learning.
- **Regularization:** Should permit regularization to steer exploration towards desirable behaviors.
- **Time-Independence:** Policies should generalize across time, enhancing robustness to delays and perturbations.
- **Embodiment-Agnosticism:** The policy should be transferable across robot morphologies, avoiding hardcoded joint trajectories.

- **Invariance:** It should capture task invariances (e.g., rotation, scale, translation), allowing better generalization.
- **Correlations:** The representation should naturally encode control signal correlations, akin to motor synergies in animals.
- **Globality:** It should help the RL algorithm escape local minima and search the global landscape effectively.
- **Periodicity:** The policy must represent periodic motions easily, such as locomotion or repetitive manipulation tasks.
- **Analyzability:** Should support interpretability and formal analysis (e.g., stability checks).
- **Multi-Dimensional Feedback:** Capable of utilizing multi-dimensional sensor feedback without collapsing it into scalars.
- **Convergence:** The policy encoding should facilitate faster and more stable convergence of the learning process.

Note

Designing a policy representation that satisfies all these criteria is extremely difficult. Most state-of-the-art representations address only a subset of these challenges—necessitating careful design choices depending on the specific robotic task and setup.

12 Perspectives on Sim2Real Transfer for Robotics

This section summarizes the key discussions, debates, and takeaways from the Sim2Real workshop held at the 2020 Robotics: Science and Systems (R:SS) conference. The event brought together a diverse group of researchers to reflect on the status and future of Sim2Real transfer in robotics, its practical importance, and how to improve simulation-to-reality performance.

12.1 Core Questions and Debates

Three main debates structured the workshop:

1. Why Sim2Real? Proponents argue that simulation provides safety, faster development cycles, and scalability at lower cost. It enables researchers to explore a wide range of policies before deploying them in the real world. Critics counter this by claiming that Sim2Real often results in *Sim2Null*, where policies learned in simulation do not transfer successfully to real robots. This debate highlighted the need for honest evaluations of transferability and the risk of overfitting to simulation.

2. Is Sim2Real a Field of Its Own? Some argue that Sim2Real is not a separate research field but an intersection of methods from domain adaptation, transfer learning, system identification, and reinforcement learning. Others believe that the specific challenges of bridging simulation and reality justify Sim2Real as a distinct area, especially due to the need to integrate physics-based simulation, perception, control, and learning.

3. How Accurate Must Simulations Be? A key tension discussed was whether simulators must be highly accurate or just "good enough" to support learning transferable policies. Some researchers emphasized improving simulation fidelity, while others supported using techniques like domain randomization and high-level abstractions that do not rely on perfect physical models.

12.2 Panel Discussion Insights

The panel emphasized the dichotomy between *task-specific* and *task-agnostic* simulation. The importance of differentiable simulators, meta-learning, and system identification for improving transfer was repeatedly mentioned. A recurrent theme was the risk of focusing too much on *Sim2Sim* validation, i.e., evaluating methods only in simulation instead of testing on real hardware. There was a collective call for deeper empirical grounding and real-world benchmarks.

12.3 Research Contributions

Eighteen short paper presentations were grouped into three themes:

- **Understanding Sim2Real Difficulty:** Several works tackled the formalization of Sim2Real and proposed metrics such as SRCC (Sim2Real Correlation Coefficient) to evaluate transferability. Other contributions focused on the challenges of simulating contact-rich dynamics.
- **Bridging the Reality Gap:** This included techniques like domain randomization, intermediate representation learning (e.g., depth maps, semantic segmentation), and visual domain adaptation using GANs. Meta-learning approaches were also proposed to increase generalization to unseen environments.
- **Learning Better Models:** Some contributions focused on residual models that compensate for simulator inaccuracies, differentiable simulators to enable gradient-based policy

learning, and hybrid approaches that combine simulation data with real-world experience. Model identification and curriculum learning strategies were also highlighted as promising solutions.

12.4 Conclusions and Future Directions

The workshop concluded with a strong affirmation of Sim2Real as one of the most promising and fast-moving areas in robotics. Despite current limitations, several methods have already demonstrated successful deployment on real robots. Future priorities identified include:

- More effective domain randomization strategies;
- High-fidelity simulation of contact dynamics and interaction;
- Use of differentiable physics engines for end-to-end learning;
- Scalable methods for real-world deployment;
- Shared benchmarks and evaluation frameworks for Sim2Real research.

Ultimately, Sim2Real is not just about simulation fidelity—it requires bridging the conceptual and practical gap between learning in a controlled simulated world and operating in a complex, unpredictable reality. The field will continue to evolve as a collaborative effort between machine learning, control theory, simulation, and robotics communities.

13 Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World

This section summarizes the paper *"Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World"* by Josh Tobin et al.

13.1 Motivation

Training robots in the real world is expensive, time-consuming, and often impractical. Simulations provide a scalable and safe alternative, but transferring behaviors from simulation to reality is hindered by the so-called *reality gap*—discrepancies in visual realism, physics, and sensor noise.

13.2 Key Idea: Domain Randomization

The central idea of this work is **domain randomization**, where the model is trained in simulation under extensive variability, such as:

- Random textures (solid colors, gradients, checkerboards)
- Varying lighting conditions and positions
- Random camera viewpoints and object locations
- Random numbers and placements of distractor objects

The hypothesis is that sufficient variability during training enables generalization to the real world, which appears simply as another domain variation.

13.3 Method

The authors trained a convolutional neural network (CNN) based on the VGG-16 architecture to localize geometric objects using only synthetic RGB images generated with MuJoCo. Notable aspects of the approach include:

- No real-world images are used for training
- No ImageNet pretraining required—random initialization sufficed with enough training data
- Cluttered scenes were used during training to promote robustness

13.4 Results

- Achieved **1.5 cm** localization accuracy in real-world scenes with occlusions
- Ablation studies showed the importance of texture diversity, camera perturbations, and distractors
- Successfully transferred to real-world manipulation tasks using a Fetch robot, including cluttered scenes
- Custom object detector (trained only on geometric proxies) generalized well to real-world objects like a Spam can

13.5 Conclusion and Future Work

This paper demonstrates that deep neural networks trained exclusively on synthetic images can transfer directly to real robotic tasks. Future directions include:

- Using higher-resolution inputs
- Incorporating additional sensor modalities (e.g., depth)
- Applying the technique to reinforcement learning scenarios
- Combining with domain adaptation methods

Domain randomization presents a promising approach to overcoming the reality gap, especially for deep learning-based control in robotics.

14 Sim-to-Real Transfer of Robotic Control with Dynamics Randomization

This paper presents a method for transferring control policies trained in simulation to real-world robots by leveraging **dynamics randomization**. The central idea is to train a policy across a wide range of randomized simulation dynamics, enabling the policy to generalize well to real-world variability without any real-world training.

14.1 Motivation

Deep reinforcement learning (RL) has shown impressive results in simulation, but transferring those capabilities to the physical world is challenging due to the "reality gap" — the mismatch between simulated and real-world dynamics. Traditional high-fidelity simulation requires meticulous calibration and is often insufficient to ensure transfer. Dynamics randomization provides an alternative by intentionally introducing variability during training.

14.2 Method

The authors demonstrate their approach on a **puck-pushing task** using a 7-DOF Fetch Robotics arm. The policy is trained entirely in the MuJoCo simulator with randomized parameters including:

- Physical properties (mass, damping, friction, table height)
- Actuation characteristics (controller gains, action timestep)
- Sensor noise (Gaussian noise on state observations)

This results in a total of 95 parameters being randomized during training. The policy is represented using a recurrent neural network (LSTM) to maintain an internal memory of past interactions, allowing it to implicitly infer the dynamics of the environment (i.e., perform system identification).

14.3 Training Approach

The training uses **Hindsight Experience Replay (HER)** with a sparse binary reward: the agent receives 0 if the puck reaches the target area, and -1 otherwise. **Recurrent Deterministic Policy Gradient (RDPG)** is applied to support off-policy learning and leverage HER.

14.4 Architecture

The policy and value networks both include:

- A recurrent branch (LSTM with 128 units) that processes previous states and actions.
- A feedforward branch for current state and goal.
- Outputs: actions from $\pi(s_t, z_t, g)$ and values from $Q(s_t, a_t, y_t, g, \mu)$.

The architecture supports memory-based adaptation to variable dynamics without explicitly estimating parameters.

14.5 Results

Policies trained with dynamics randomization performed robustly on the real robot without any additional training. Key findings include:

- LSTM-based policies achieved high success rates (89% on real robot).
- Feedforward policies without randomization failed completely in the real world.
- Randomization of specific parameters (action timestep and observation noise) was crucial for successful transfer.
- The policy remained robust even when contact dynamics were altered by attaching a bag of chips to the puck.

The paper also compares network architectures, showing that recurrent models outperform memoryless ones, especially under real-world deployment conditions.

14.6 Conclusion

Dynamics randomization is an effective strategy for bridging the sim-to-real gap in robotic learning. By training across a diverse range of simulated dynamics, policies become robust to discrepancies and calibration errors in real hardware. This approach enables powerful RL algorithms to be trained entirely in simulation and deployed directly to physical systems with minimal performance loss.

15 Pretraining Language Models with Human Preferences

This section summarizes the paper titled *"Pretraining Language Models with Human Preferences"* (arXiv:2111.00956v2), which presents a framework for improving large language models (LLMs) through the integration of human feedback. The main goal is to fine-tune models in a way that their behavior more closely aligns with human expectations, enhancing both usefulness and safety.

15.1 Motivation and Background

Traditional supervised fine-tuning methods are limited by the quality and scope of their training data. While large-scale language modeling produces fluent and factually relevant outputs, such models may generate unsafe or unhelpful responses. Human preferences, when used as a guide for optimization, can bridge this gap by providing direct feedback on the desirability of model outputs.

15.2 Key Contributions

- **Reinforcement Learning from Human Feedback (RLHF):** The central contribution is the use of reinforcement learning to fine-tune language models based on a reward model trained from human-labeled data. This method allows the model to better reflect human preferences in its output.
- **Three-Stage Training Pipeline:**
  1. **Supervised Fine-Tuning (SFT):** The base LLM is first fine-tuned using a curated dataset of high-quality, human-written responses.
  2. **Reward Modeling (RM):** Human labelers compare outputs from different models. These preferences are used to train a reward model that estimates the quality of a model output.
  3. **Reinforcement Learning (PPO):** The model is further fine-tuned using Proximal Policy Optimization (PPO), where the objective is to maximize the score predicted by the reward model.
- **Evaluation Results:** The models fine-tuned with RLHF perform significantly better on human evaluation metrics of helpfulness, honesty, and harmlessness compared to their SFT-only counterparts. Notably, they generalize better to unseen prompts and avoid producing certain classes of problematic content.
- **Insights on Reward Hacking:** A known challenge is that optimizing solely for the reward model can lead to undesirable behaviors if the model learns to exploit biases or flaws in the reward function. The paper discusses mitigation strategies, including careful curation of human feedback and robustness testing.

15.3 Limitations and Challenges

- **Imperfect Reward Models:** Human labelers may be inconsistent or biased, and the reward model may fail to generalize well to all scenarios.
- **Overoptimization Risks:** There is a risk of the model producing outputs that game the reward model rather than reflecting true alignment with human preferences.
- **Scalability:** Human feedback collection is expensive and difficult to scale, which limits the diversity and size of datasets for reward modeling.

15.4 Future Directions

- Explore more robust and generalizable reward modeling techniques.
- Incorporate diverse and representative human feedback to reduce bias.
- Combine RLHF with other alignment techniques, such as Constitutional AI or preference elicitation methods.
- Develop better interpretability tools for understanding the behavior of reward-optimized models.

15.5 Conclusion

This paper represents a major step forward in aligning language model behavior with human preferences. By combining supervised learning, reward modeling, and reinforcement learning, the approach achieves improved outcomes in safety, usefulness, and adherence to user intent. Despite its limitations, the RLHF framework has already influenced the training pipelines of major LLMs and continues to serve as a cornerstone in the field of AI alignment.

16 Generalizing from Simulation (OpenAI)

This section summarizes the key points from the OpenAI article *"Generalizing from Simulation"*, which presents a method for training robots entirely in simulation and successfully transferring them to operate in the real world.

16.1 Core Idea

The work introduces **dynamics randomization** as a technique to close the gap between simulation and reality. By introducing large variability into the simulated environment during training, policies become robust to discrepancies and uncertainties present in the real world.

16.2 Key Techniques

- **Dynamics Randomization:** Physical parameters such as mass, friction, damping, actuator delays, and sensor noise are randomized during training. This exposes the agent to a wide range of possible conditions.
- **Closed-Loop Control:** Unlike open-loop control (pre-planned actions), the robot uses closed-loop control to continuously adjust its behavior in response to real-time observations.
- **LSTM Networks:** Policies are implemented using Long Short-Term Memory (LSTM) neural networks, allowing the robot to retain past information and infer hidden dynamics during operation.

16.3 Experiments and Results

- **Puck Pushing Task:** A robot trained entirely in a randomized simulator was able to perform puck pushing in the real world with high success, adapting to unseen physical properties.
- **Vision-Based Policy Transfer:** The team also trained an end-to-end vision-based control policy using reinforcement learning. The robot was able to act using only visual inputs, without relying on special instrumentation.

16.4 Implications

This research demonstrates that simulation-trained policies can generalize effectively to real-world scenarios without additional fine-tuning, as long as appropriate variability is introduced during training. This drastically reduces the cost and effort associated with real-world training and paves the way for scalable, adaptable robotic systems.

Note

This work represents a significant step toward general-purpose robots that can learn robust behaviors in simulation and perform reliably in the unpredictable real world.

17 Randomization and the Reality Gap: Improving Policy Transfer in Sim-to-Real Reinforcement Learning

17.1 Overview

One of the central challenges in applying reinforcement learning (RL) to robotics is the difficulty of transferring policies learned in simulation to the real world. This is known as the *reality gap* — the difference in dynamics, observations, and noise between simulation and physical environments. The paper by Josh Tobin et al. investigates how **domain randomization**, a technique that introduces variability in the simulated training environment, can bridge this gap effectively.

17.2 Motivation and Problem Statement

Reinforcement learning has achieved remarkable success in simulated environments, but transferring these learned policies to real robots often fails due to discrepancies in:

- Physical properties (e.g., mass, friction)
- Sensor and actuator noise
- Latencies and delays
- Model simplifications in the simulator

These discrepancies can lead to brittle policies that do not generalize beyond the specific simulation they were trained in.

17.3 Domain Randomization Approach

The core idea of domain randomization is to expose the agent to a wide variety of simulated environments during training. By varying key parameters in each training episode, the policy is encouraged to learn behaviors that are robust across a distribution of environments rather than a single, fixed one.

The parameters randomized include:

- Mass and friction coefficients
- Motor strengths and latencies
- Observation noise
- Delay in sensor readings
- Environmental textures and lighting (for vision-based tasks)

17.4 Experiments

Simulation Environment: The authors use the MuJoCo physics engine to simulate locomotion tasks, training policies on a quadrupedal robot model called the *Minitaur*.

Transfer to Reality: Policies trained in the randomized MuJoCo simulator are transferred to a real-world Minitaur robot. The evaluation focuses on tasks such as:

- Walking straight
- Turning in place

17.5 Findings and Insights

- **Domain randomization enables successful transfer:** Policies trained with randomized parameters consistently outperformed those trained in fixed environments when deployed in the real world.
- **Importance of latency and noise:** Among all randomized parameters, introducing sensor noise and actuator latency was most critical for closing the reality gap.
- **Curriculum learning helps:** Gradually increasing the degree of randomization over time — rather than applying full randomization from the beginning — improves training stability and final policy performance.
- **Over-randomization can be harmful:** Excessive variability, especially early in training, can make learning unstable or ineffective. The distribution of randomized parameters must be chosen carefully.
- **Generalization improves:** Domain-randomized policies generalize better to unseen test environments, including real-world terrain with dynamics not seen during training.

17.6 Practical Recommendations

- Identify and randomize the parameters that are most likely to differ between sim and real environments.
- Use a curriculum-based approach to progressively increase randomization.
- Avoid unnecessary randomization of non-impactful parameters to maintain learning stability.
- Combine domain randomization with robust policy representations and reward shaping.

17.7 Conclusion

Domain randomization is a simple yet powerful technique to mitigate the reality gap in reinforcement learning. It enables successful transfer of control policies from simulation to real robots, even in high-dimensional and dynamic tasks. When applied thoughtfully, domain randomization can significantly increase the robustness and real-world applicability of learned behaviors, paving the way for more widespread use of RL in robotics.

18 Domain Randomization for Sim2Real Transfer (Lilian Weng, 2019)

18.1 Motivation

Reinforcement learning models trained in simulation often fail when deployed on physical robots due to the **reality gap**—discrepancies in physical properties, sensor behavior, and observation noise. Since real-world data collection is expensive, domain randomization (DR) aims to improve transfer by exposing the policy to a diverse range of simulated environments.

18.2 Sim2Real Transfer Strategies

Three main approaches are discussed:

- **System Identification:** Precisely calibrate simulators to match real-world physics. While accurate, it is expensive and sensitive to hardware variability.
- **Domain Adaptation (DA):** Uses real data to bridge the gap between simulated and real distributions, often with GANs or adversarial methods.
- **Domain Randomization (DR):** Randomizes visual and physical simulation parameters so the real world becomes just another variation within the training distribution.

18.3 What is Domain Randomization?

Domain randomization defines a space of configurations $\xi \in \Xi$ for the simulator. During training, ξ is sampled to generate randomized environments. The policy π is trained to maximize the expected return over the distribution ϕ:

$$\pi^* = \arg\max_{\pi} \mathbb{E}_{\xi \sim \phi} [R(\tau_\xi)]$$

where τ_ξ is a trajectory collected in a simulator instance randomized by ξ.

18.4 Uniform Domain Randomization (UDR)

In UDR, each parameter is uniformly sampled from predefined intervals. Examples include:

- **Visual:** Object shape, color, texture, lighting, camera pose
- **Physical:** Mass, damping, friction, PID gains, joint limits, latency, observation noise

OpenAI used DR to train a dexterous robotic hand to manipulate an object into 50 random target orientations. Initially unstable, the policy eventually succeeded in real-world trials.

18.5 Why Domain Randomization Works

1. DR as Optimization Domain randomization can be viewed as a bilevel optimization problem. The goal is to find the optimal randomization distribution ϕ so that a policy π trained in randomized environments performs well in the real world:

$$\phi^* = \arg\min_{\phi} \mathcal{L}_{\text{real}}(\pi^*_\phi) \quad \text{with} \quad \pi^*_\phi = \arg\min_{\pi} \mathcal{L}_\phi(\pi)$$

2. DR as Meta-Learning When using memory-based policies (e.g., LSTMs), the agent can dynamically adapt to different environmental configurations, effectively learning how to learn across a range of tasks.

18.6 Guided Domain Randomization

Rather than using uniform sampling, guided DR seeks to optimize ϕ using feedback signals.

Strategies:

- **AutoAugment-style Search (Cubuk et al.):** Treats randomization config search as a reinforcement learning problem using task performance as reward.
- **Learning to Simulate (Ruiz, 2019):** Uses performance feedback to train a policy that predicts useful ϕ values.
- **CMA-ES (Yu et al.):** Evolutionary strategy where randomization parameters evolve based on task fitness.

18.7 Real Data-Guided DR and Distribution Matching

- **SimOpt (Chebotar et al., 2019):** A policy is trained in simulation and evaluated in both sim and real-world. The randomization distribution is updated to minimize trajectory divergence:

$$\phi^* = \arg\min_{\phi} \mathbb{E}_{\tau_s \sim \pi_\phi} \left[D(\tau_s, \tau_r) \right]$$

  where D is a discrepancy metric between simulated trajectory τ_s and real-world τ_r.

- **RCAN (James et al., 2019):** Uses a conditional GAN to translate real or randomized images into a canonical simulated image for consistency in perception.

18.8 Simulator-Guided Domain Randomization

- **DeceptionNet (Zakharov et al., 2019):** Applies adversarial image transformations to make the recognition task harder, improving robustness.
- **Active Domain Randomization (ADR; Mehta et al., 2019):** Searches for randomizations that maximize behavioral divergence compared to a reference environment. The discrepancy is learned via a discriminator, and SVPG is used to explore high-impact variations.

18.9 Conclusion

Domain randomization is an effective method for sim-to-real transfer, offering a low-cost, scalable alternative to real-world data collection. While uniform DR provides a strong baseline, guided approaches can further enhance performance by focusing on impactful variations. The method integrates well with RL and continues to be central in real-world robotics research.